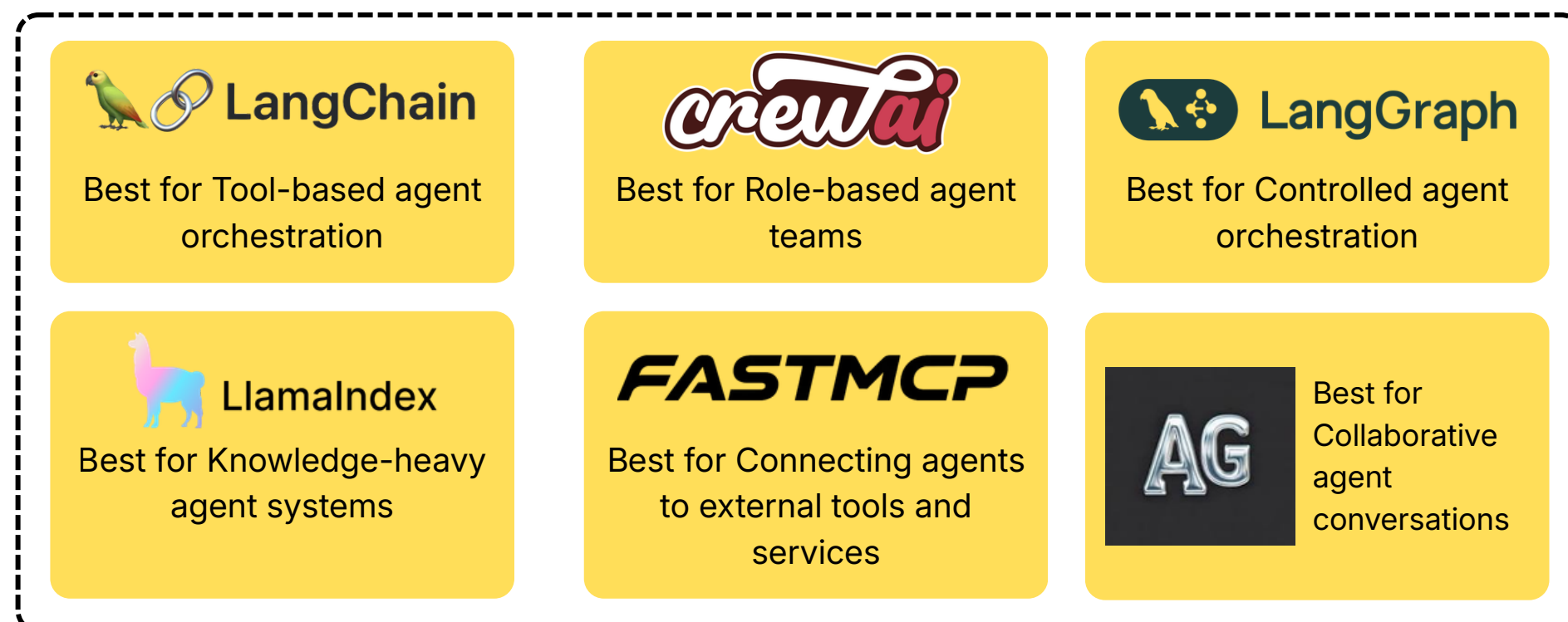
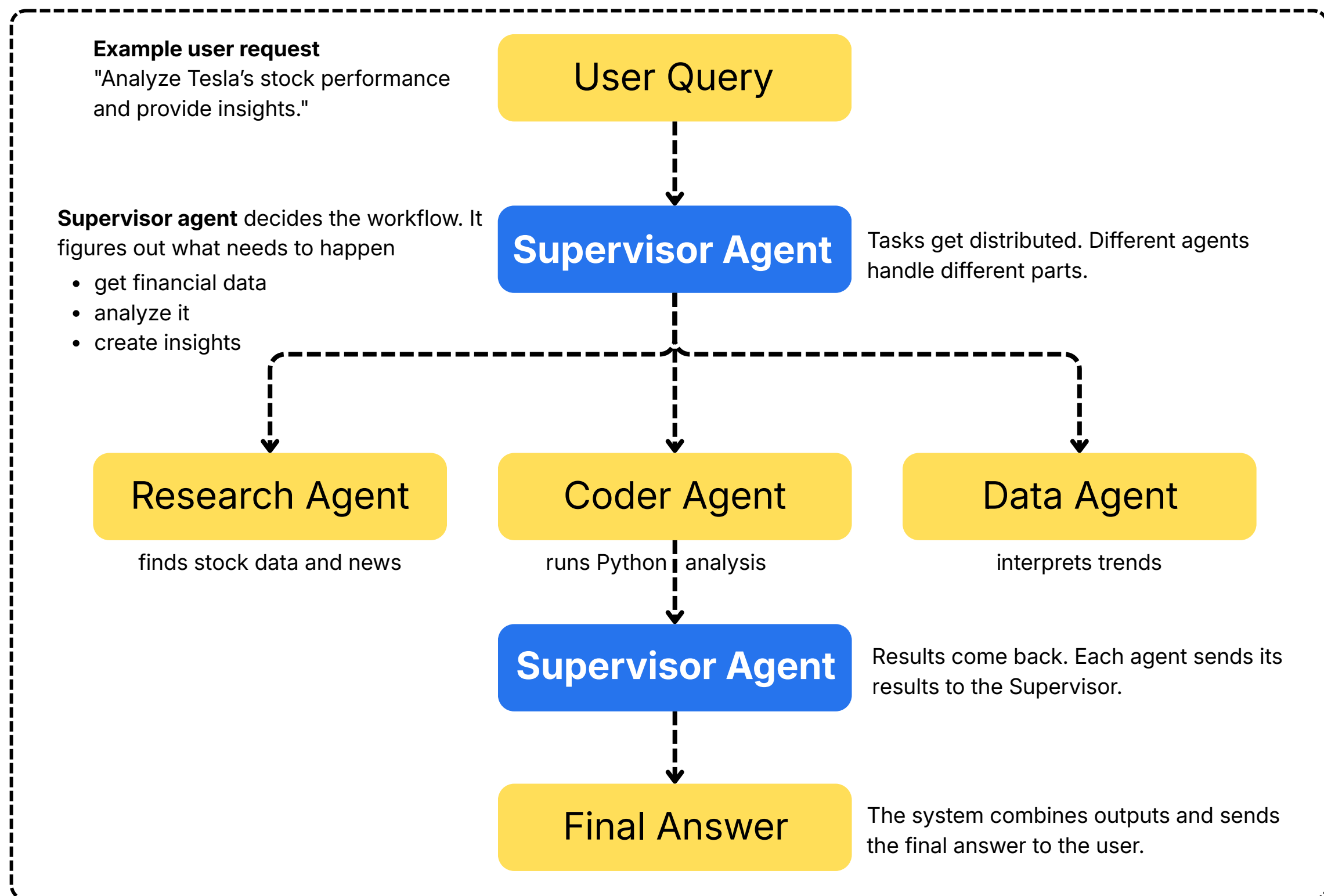


A Guide to Multi Agent System Design Patterns



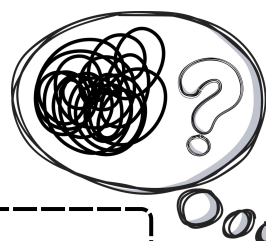
The Problem



Start with one agent

ONE AGENT

when possible



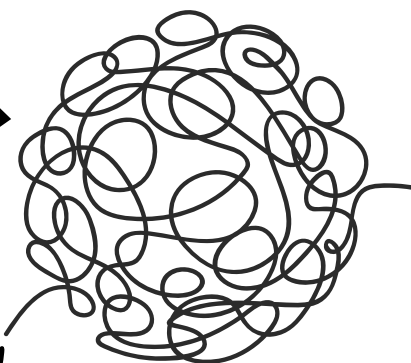
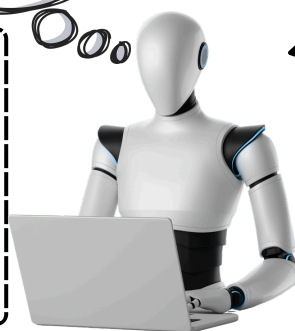
A single agent struggles with:

Long reasoning chains

Too much context

Too many tools

Mixed responsibilities



But as
complexity
grows



This is where

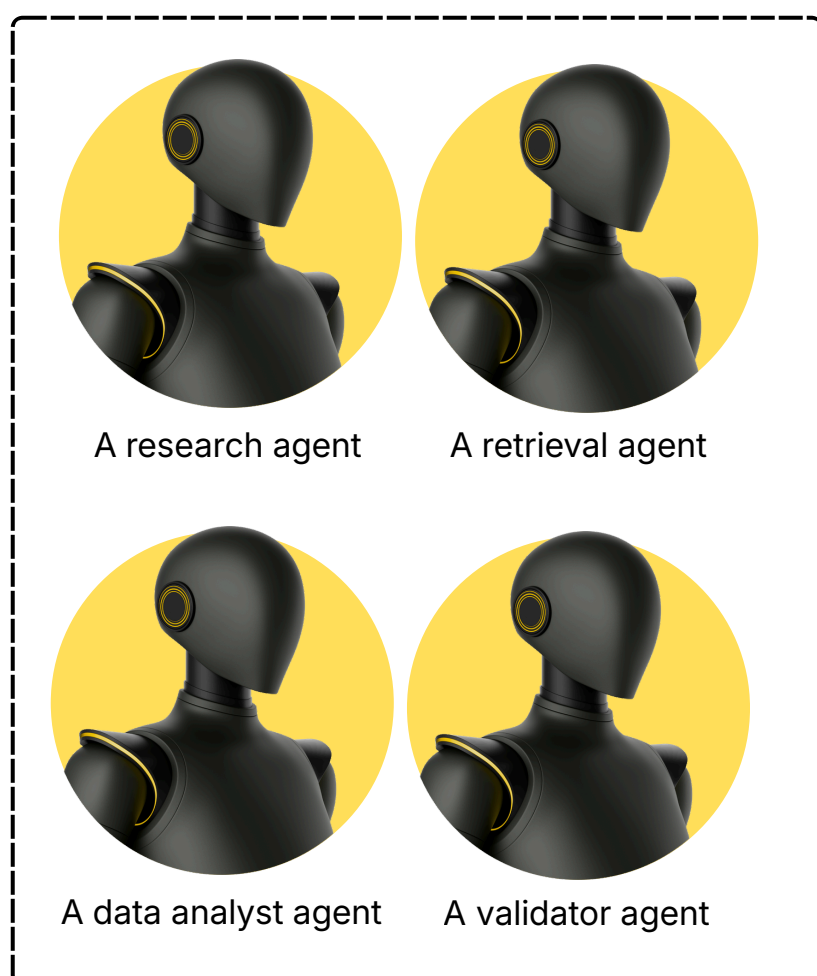
**Multi
Agent
Systems**

come in

Think of them as **teams of
specialized AI agents**

What is a Multi-Agent System?

A multi-agent system is an AI architecture where: Multiple specialized **agents collaborate to solve a task**. Instead of one generalist AI, you have:

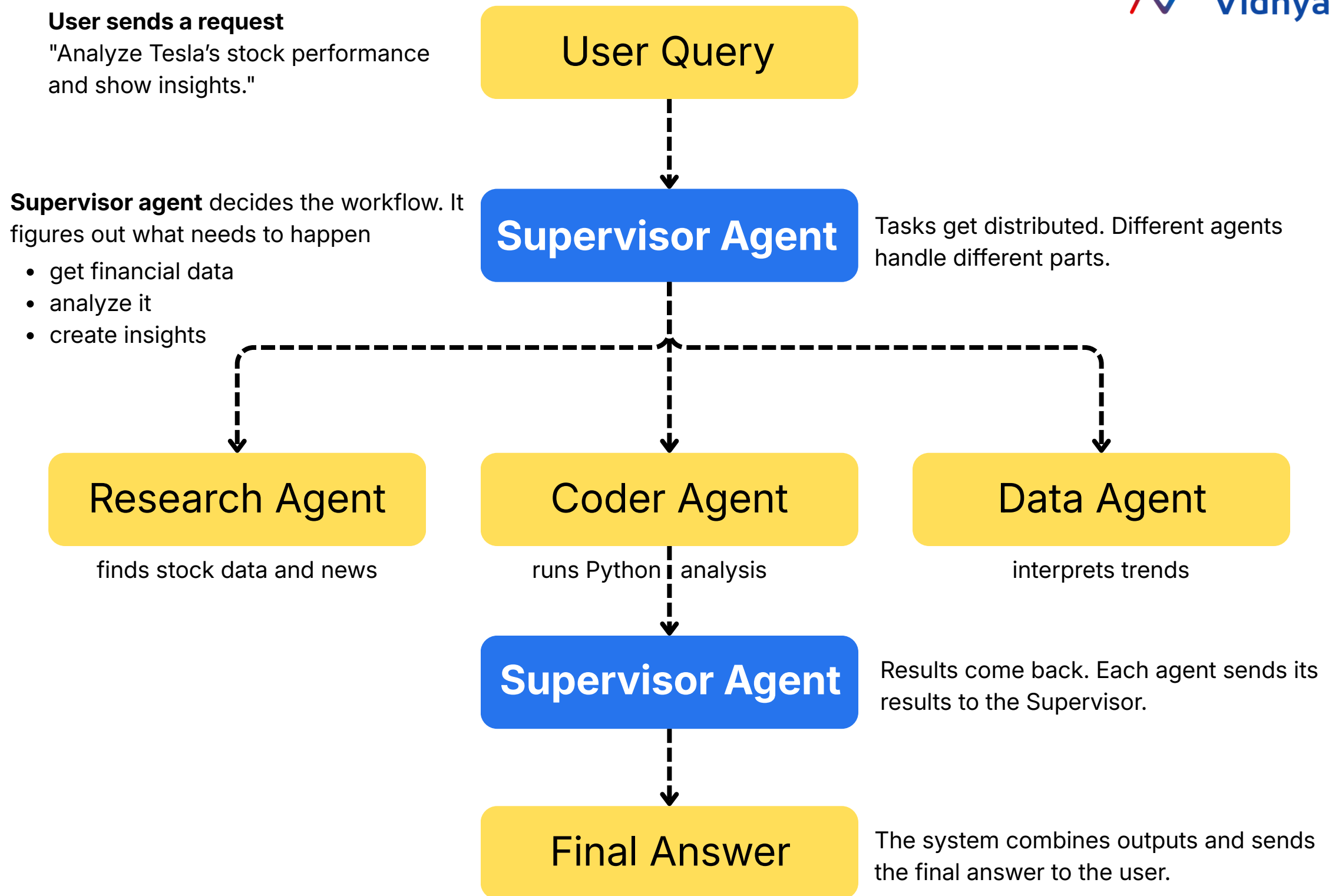


Each agent focuses on one capability.

Just like teams in real organizations.

Instead of one large agent doing everything, tasks are divided among agents with specific roles, tools, and expertise.

Aspect	Tool-Based System	Multi-Agent System
Basic Idea	One main agent (LLM) decides when to call external tools or functions	Multiple specialized agents collaborate to solve a task.
Structure	Single decision-maker + tools (APIs, databases, calculators, etc.)	Multiple decision-makers with different roles
Control Flow	Centralized – one agent orchestrates everything.	Distributed – agents communicate and coordinate.
Complexity	Simpler to build and maintain	More complex due to coordination between agents.
Typical Roles	One agent using tools like search, code execution, retrieval	Planner agent, researcher agent, coder agent, critic agent, etc.
Example Use Case	Chatbot that calls weather API or database	System where one agent plans, another gathers info, another evaluates results



Instead of one AI trying to:

search → code → analyze → explain

You have multiple specialized agents doing their best job.

This improves:

- reliability
- reasoning
- scalability
- maintainability

Core Principles of Multi-Agent Systems

1

Distributed Intelligence

Multiple smaller agents handle tasks instead of one giant model.

2

Defined Roles

Each agent has a specific responsibility or expertise.

3

Collaboration

Agents communicate with each other to exchange information.

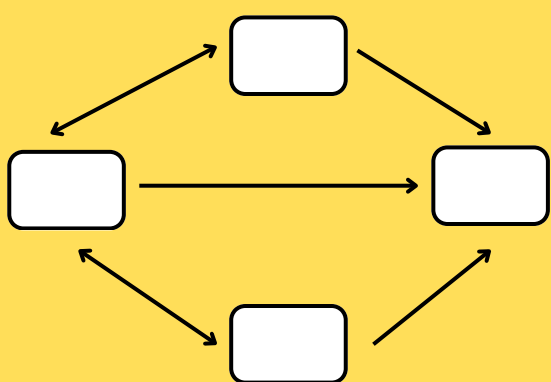
4

Orchestration

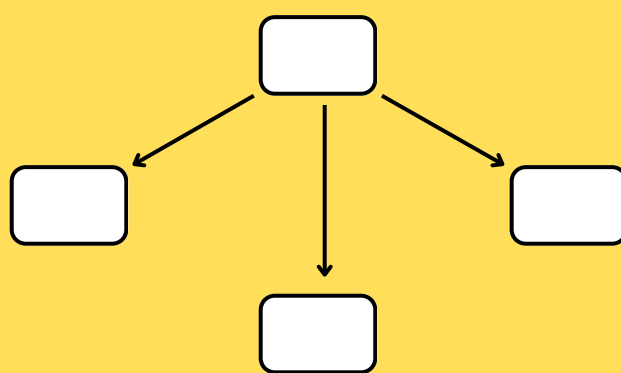
A supervisor agent or coordination logic decides how tasks flow across agents.

Major Multi-Agent Design Patterns

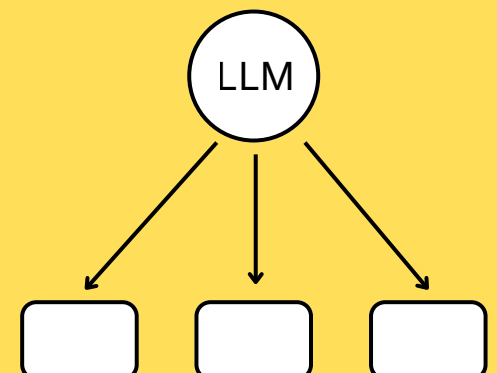
Network Architecture



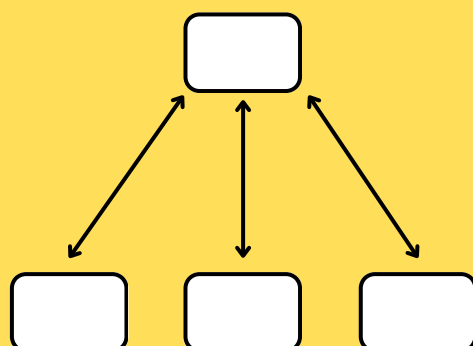
Supervisor Architecture



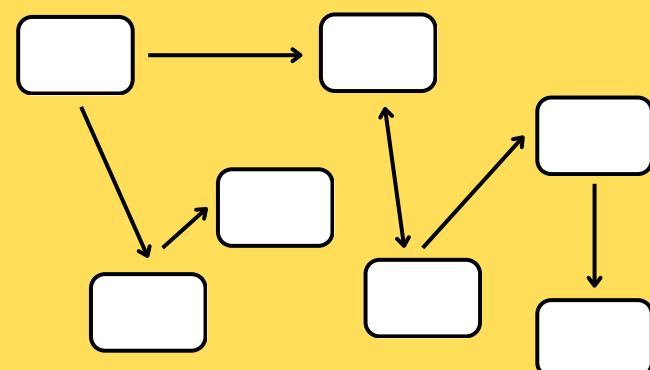
Supervisor (Tool Calling)



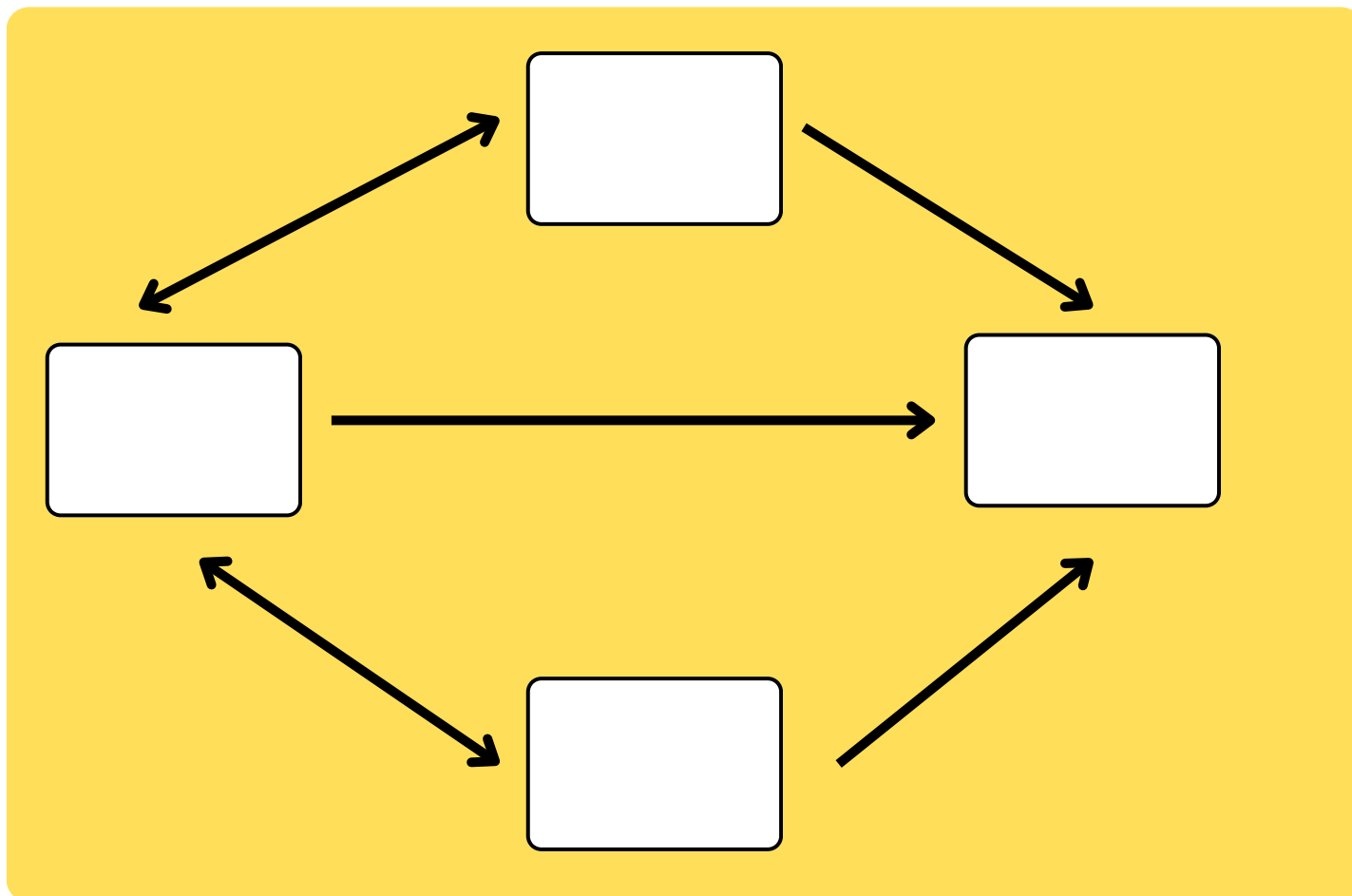
Hierarchical Architecture



Custom Architecture



Network Architecture



In the network pattern, every agent can communicate directly with every other agent.

Features:

- peer-to-peer collaboration
- decentralized decision making
- agents choose which agent to call next

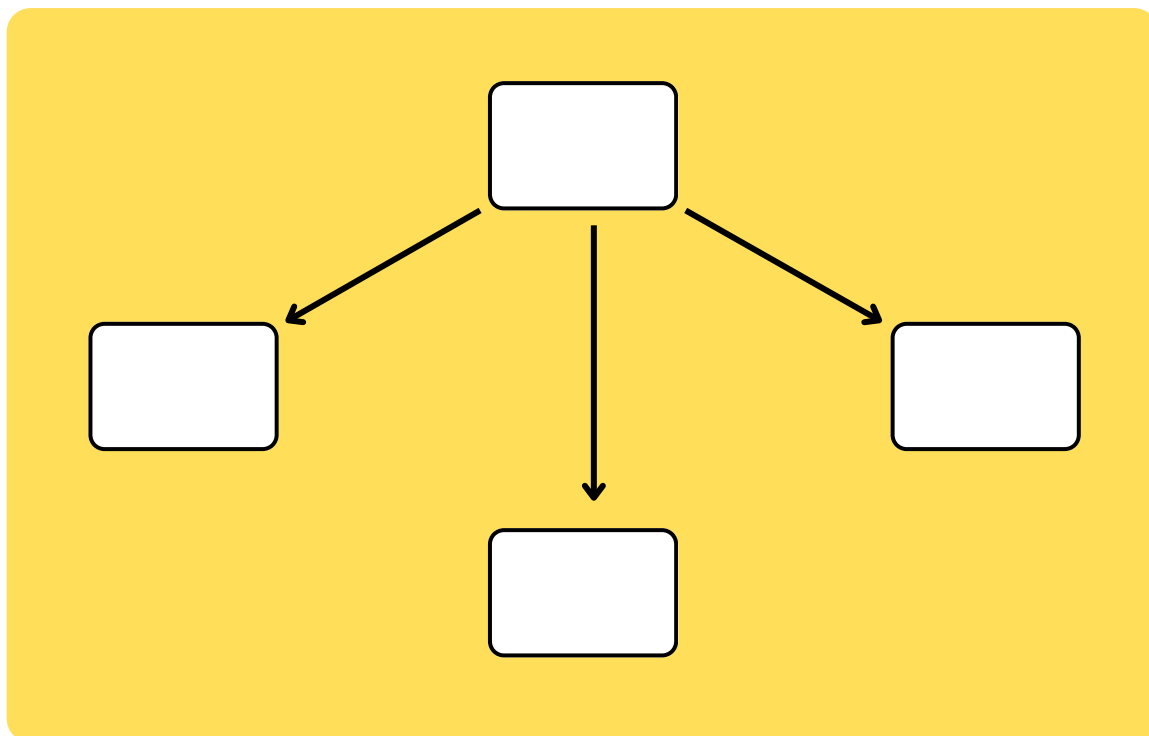
Best for:

- collaborative reasoning systems
- research assistants
- brainstorming agents

Downside:

Harder to control and monitor workflows.

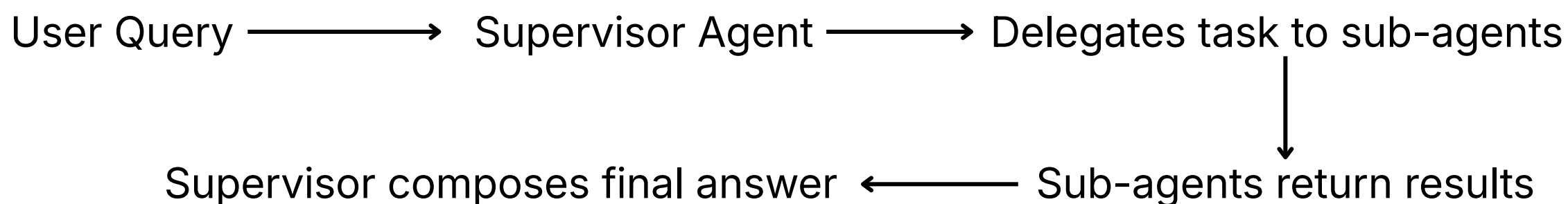
Supervisor Architecture



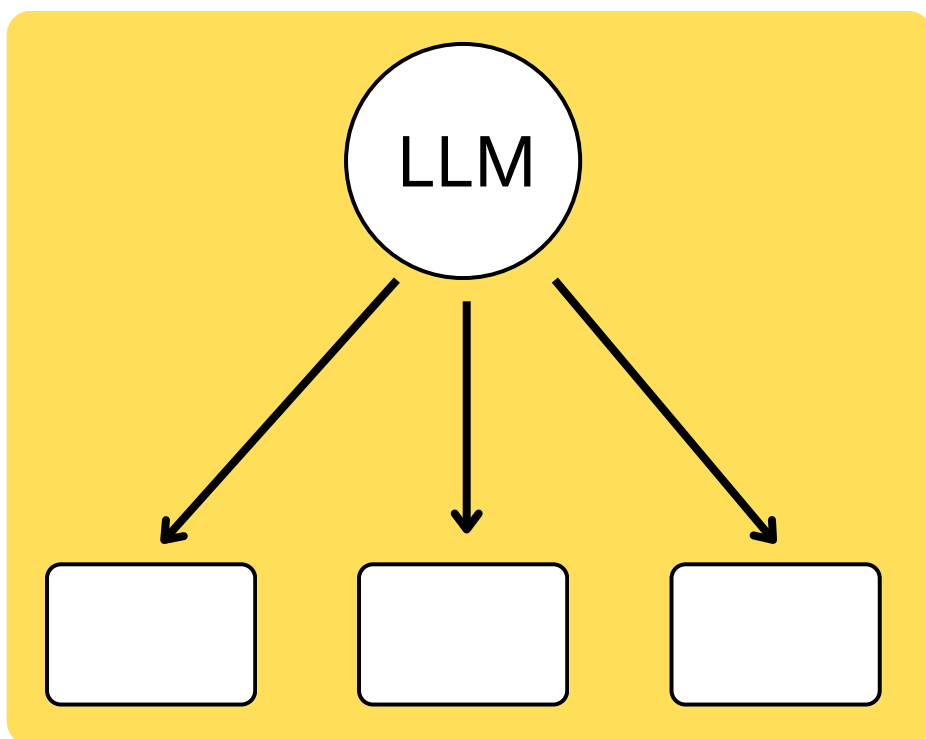
This is one of the most widely used multi-agent patterns. A central supervisor agent orchestrates all other agents.

Benefits:

- clear task routing
- easier monitoring
- controlled workflow



Supervisor Tool-Calling Architecture



A variation of supervisor architecture. In this pattern, Agents are exposed as callable tools.

The supervisor LLM decides:

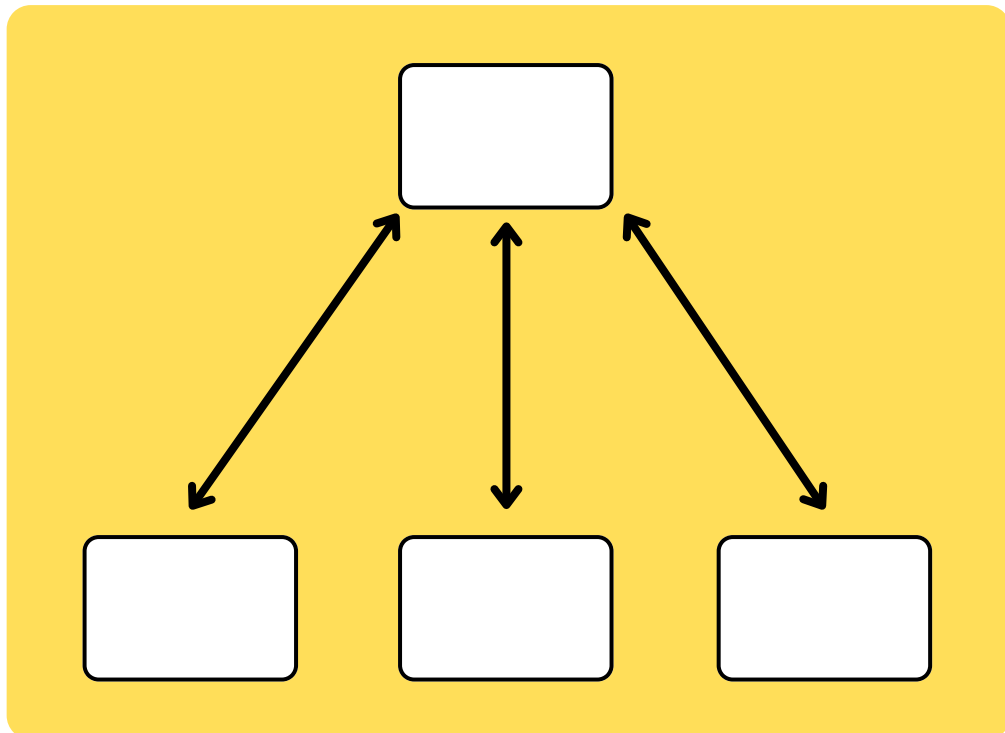
- Which agent tool to call.
- What arguments to pass.
- When to call the next agent.

This pattern works very well with function-calling LLM APIs.

Think of it as:

LLM orchestrator + Agents as callable functions

Hierarchical Architecture



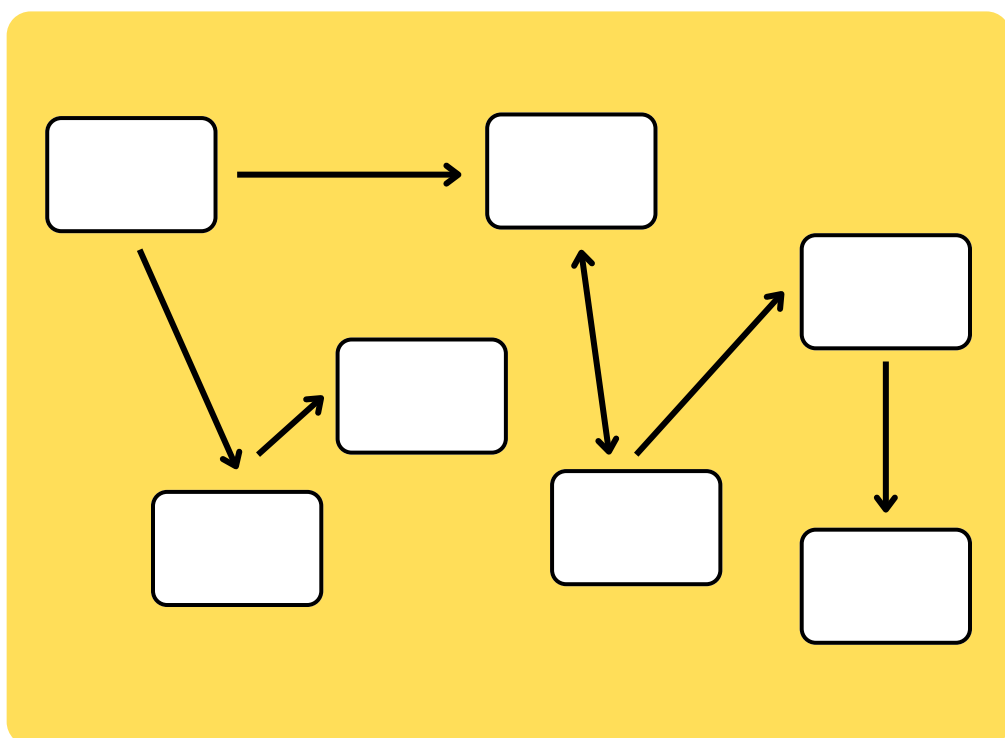
Used for large and complex systems. Each supervisor manages a group of agents responsible for specific tasks. Example:

Main supervisor

- Research supervisor
- Engineering supervisor
- Data supervisor

Best for enterprise-scale agent systems.

Custom Architecture



Real-world systems often combine multiple architectures.

In custom architectures:

- agents communicate only with specific agents
- workflows combine deterministic pipelines and agents
- some processes remain rule-based

This allows high flexibility for domain-specific systems.

Example pipeline:

Research Agent

- Analysis Agent
- Visualization Agent
- Report Generator Agent

Popular Libraries for Building Multi-Agent Systems

Libraries	Best for	Strength	Use Cases
 LangGraph	Controlled agent orchestration	Graph-based workflow control	Production multi-agent pipelines
	Role-based agent teams	Simple team-style agent collaboration	research + analysis agent teams
 LangChain	Tool-based agent orchestration	Large ecosystem of integrations and tool-calling support	Single-agent systems that call tools, or simple multi-agent workflows
 LlamaIndex	Knowledge heavy agent systems	Deep integration with RAG and data sources	document analysis agents
FASTMCP	Connecting agents to external tools and data	MCP protocol for tool interoperability	tool-integrated agent systems
	Collaborative agent conversations	Built-in support for multi-agent dialogue and coordination	Research agents, coding assistants, or agents that debate and refine answers